

A* Algorithm: Finding an Optimal Path from Point A to Point B

Jason Jiang

1 Introduction

Humans have had to figure out the best route from Point A to Point B throughout history. Having been born and raised in Brooklyn, I've never been scared by a lost tourist approaching me on the streets. It's easy for us humans to retrace a previous path and guide others in the right direction.

Fast forwarding to the 21st century, the computers all around us can replicate this optimization. It's found in our video games' characters who move to our cursors, our self-driving cars, and even our vacuums. The best-known example is probably Google Maps, which gives extremely fast and accurate travel information for any route. There are many methods to solve this problem, but I will be diving into just one of the ways a computer can find an optimal path, the A* Algorithm.

2 History and Uses

A* Algorithm was created by the efforts of Peter Hart, Nils Nilsson, and Bertram Raphael back in 1968. It was first used to build faster pathfinding AI for a robot called Shakey the Robot. The implications of their discovery were not just limited to robotics. Pathfinding algorithms are useful for sequence games like Othello. They are also used in path movement for video games that move the player to the cursor. They help with computer traffic and graph traversal. Their applications are incredibly varied! Anywhere that you have different paths and want to know how to get from Point A to Point B, the A* Algorithm is your best friend!

3 How it Works

A* Algorithm looks at undiscovered places to go, using a heuristic that estimates the total cost of going from the origin through the node to the goal, evaluating the most promising nodes first.

1. The A* Algorithm starts at the origin, looks at all the undiscovered points that it can reach in one move, and then adds them to a queue with the parent property as the origin and keeps track of the cost to get there.

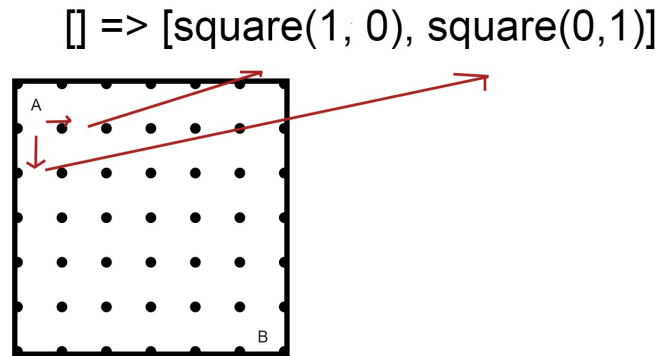


Figure 1: Step 1: The squares next to the starting point (A) are added into the queue

2. This is when the “secret sauce” of the A* Algorithm gets to work. The queue is sorted by adding together the cost to get to the node and the estimated distance from the destination. Together this forms an estimate of the distance that the most optimal path would take going through that node:

$$f(n) = g(n) + h(n)$$

f = total estimated cost to go through node

g = cost to get to the node

h = estimated distance to get to the destination

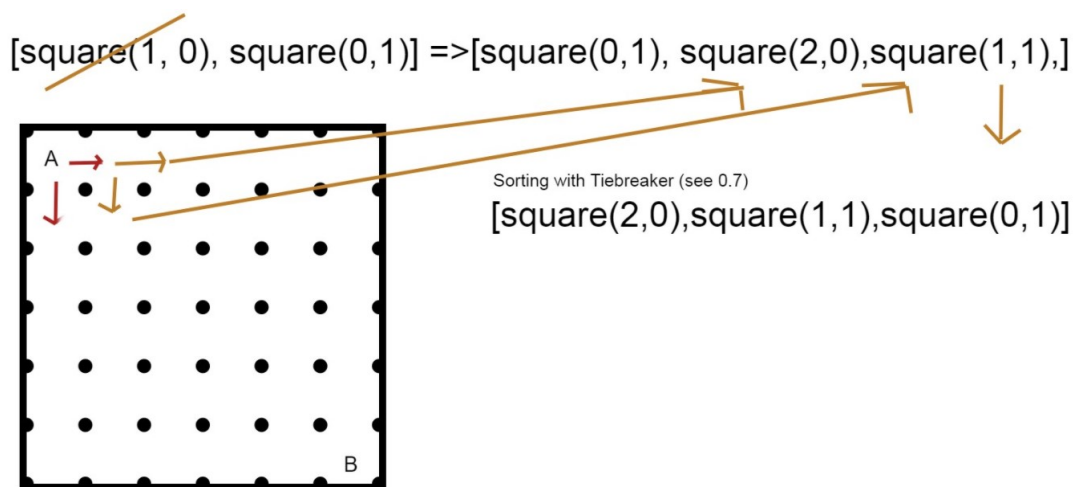


Figure 2: Step 2: From the origin, A* looks at square(1,0) and finds the nearby squares: square(2,0), square(1,1). It then appends and sorts them into the list of undiscovered squares.

3. The A* Algorithm then repeats the first two steps, looking at the undiscovered nodes around the first element in the sorted queue instead of the origin. It then continues evaluating these nodes until either the program reaches the destination or a path is not found.

4. At the end point, the program can backtrack to each of the parent nodes until it reaches the origin, constructing the solution path and calculating its distance. Every node contains a back pointer which stores the previous node in the path.

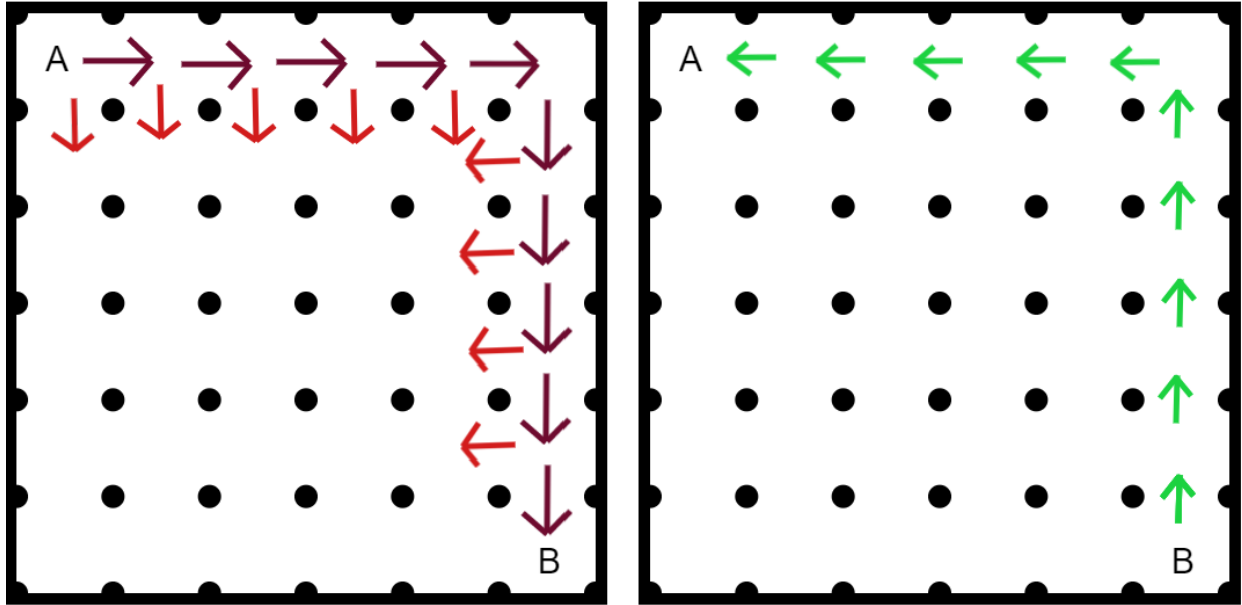


Figure 3: Steps 3 and 4: A* searches through the grid, prioritizing nodes that move closer to the destination. After reaching the end point, it retraces its steps to find the optimal path.

4 (When) does A* give us the correct shortest path?

In a specific implementation of A*, the answer to this question is: always!! To prove it, we rely on these two assumptions:

1. We have an admissible heuristic!! In the sorting process, the heuristic function estimates the cost of reaching the goal through a node. An admissible heuristic ensures that the estimation of the total cost will never be above the actual total cost, which is distance of the path drawn from the starting point, through the node, to the destination. (See the green line in visualization).
2. The heuristic value of the node is the same anywhere along its estimated optimal path. So, when A* Algorithm sorts the queue and picks the next best node, we can think of that as the A* Algorithm “trying out” this optimistic estimated path.

Thus, because A* Algorithm is always “trying out” the optimistic estimated path every time, it sorts the queue and picks the best node. We know that when A* Algorithm reaches the end point, it will have chosen the best path.

5 Manhattan Distance and Tie Breakers

A*'s performance and success is determined by the strength and accuracy of your heuristic, so having a robust one is important. You can choose from a variety of heuristics, but in my visualization, I used the Manhattan Distance, a well known heuristic that is amazing at working with grids. The formula is very simple:

$$\text{abs}(\text{goal}.x - \text{node}.x) + \text{abs}(\text{goal}.y - \text{node}.y)$$

The reasoning behind it is that no matter what, the path to reach the goal has at least a displacement of $\text{abs}(\text{goal}.x - \text{node}.x)$ and at least a displacement of $\text{abs}(\text{goal}.y - \text{node}.y)$.

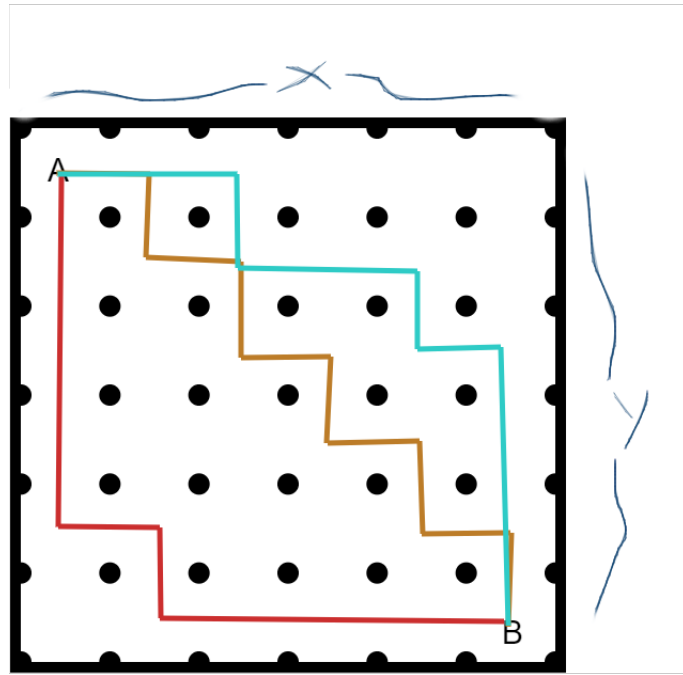


Figure 4: All these paths have the same Manhattan Distance

However, as we see in Figure 4, there can be many paths that all have the same Manhattan Distance. If they all have the same Manhattan Distance, this slows down A*, since it attempts to go down all these paths. We need some way to tell A* to prioritize just one of these equivalent paths and stick with it.

That's when tie breakers come in. To implement a tie breaker, we will create a conditional. If there is a tie among the estimated cost between nodes, A* will favor the nodes closest to the goal. Therefore, A* will not waste time exploring different but equally optimal paths.

6 Conclusion

A* is a simple but powerful pathfinding algorithm that is used in many kinds of applications. I hope this short introduction inspires you to utilize A* for your next project. Next time you flick on your phone to turn on Google Maps, pat yourself on the back because you can now appreciate some of the optimizing behind calculating your route.

References

- “A* (A Star) Search Algorithm - Computerphile.” YouTube. YouTube, February 15, 2017.
<https://www.youtube.com/watch?v=ySN5Wnu88nE>.
- Chiang Mike. A* optimality proof, cycle checking. PDF file. Jan 24, 2020.
https://www.cs.cmu.edu/15281/recitations/rec2/rec2_sol.pdf
- Mayefsky, Eric, Francine Anene, and Marina Sirota. “ALGORITHMS - A*.” Strategies and Tactics for Intelligent Search. Accessed January 12, 2021.
<https://cs.stanford.edu/people/eroberts/courses/soco/projects/2003-04/intelligent-search/astar.html>.
- Moore, Andrew. A* Heuristic Search. PDF file. <http://www.cs.cmu.edu/cga/ai-course/astar.pdf>
- Patel, Amit. “Heuristics.” Red Blob Games. Accessed January 12, 2021.
<http://theory.stanford.edu/amitp/GameProgramming/Heuristics.html>.
- Patel, Amit. “Introduction to A*.” Red Blob Games. Accessed January 12, 2021.
<http://theory.stanford.edu/amitp/GameProgramming/AStarComparison.html>.
- PDF file. January 21, 2011.
<https://www.cs.ubc.ca/hutter/teaching/cpsc322/2-Search5.pdf>